

Praktikum 6: Deadlock

Informasi Praktikum

- **Mata Kuliah:** Sistem Operasi
 - **Sistem Operasi:** Linux Ubuntu
 - **Bahasa Pemrograman:** C
 - **Dosen Pengampu:** Triawan Adi Cahyanto, M.Kom
 - **Asisten Dosen:** Atidhira Habibillah dan Taqiyyuddin
-

Tujuan Praktikum

Setelah mengikuti praktikum ini, mahasiswa diharapkan dapat:

1. Memahami konsep deadlock melalui praktik langsung
 2. Membuat program yang mengalami deadlock
 3. Mendeteksi terjadinya deadlock
 4. Menerapkan teknik pencegahan deadlock
 5. Memulihkan sistem dari kondisi deadlock
-

Persiapan

Tools yang Dibutuhkan:

- Sistem Operasi Linux (Debian/Ubuntu)
- Compiler GCC
- Text Editor (nano, vim, atau gedit)
- Terminal/Command Line

Instalasi GCC (jika belum ada):

```
sudo apt update
sudo apt install gcc
sudo apt install build-essential
```

Cek Instalasi:

```
gcc --version
```

Topik 1: Memahami Deadlock Sederhana

Penjelasan:

Bayangkan dua orang yang saling bertukar buku. Orang A pegang Buku 1 dan butuh Buku 2. Orang B pegang Buku 2 dan butuh Buku 1. Keduanya menunggu tanpa ada yang mau melepas bukunya dulu. Inilah deadlock!

Deadlock dengan Mutex

File: deadlock_simple.c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Deklarasi dua mutex (seperti kunci pintu)
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

// Fungsi untuk Thread 1
void* thread1_func(void* arg) {
    printf("Thread 1: Mencoba mengambil Lock 1...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 1: Berhasil dapat Lock 1!\n");

    sleep(1); // Simulasi proses kerja

    printf("Thread 1: Mencoba mengambil Lock 2...\n");
    pthread_mutex_lock(&lock2);
    printf("Thread 1: Berhasil dapat Lock 2!\n");

    printf("Thread 1: Selesai bekerja\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

// Fungsi untuk Thread 2
void* thread2_func(void* arg) {
    printf("Thread 2: Mencoba mengambil Lock 2...\n");
    pthread_mutex_lock(&lock2);
    printf("Thread 2: Berhasil dapat Lock 2!\n");

    sleep(1); // Simulasi proses kerja

    printf("Thread 2: Mencoba mengambil Lock 1...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 2: Berhasil dapat Lock 1!\n");

    printf("Thread 2: Selesai bekerja\n");
}
```

```

pthread_mutex_unlock(&lock1);
pthread_mutex_unlock(&lock2);

return NULL;
}

int main() {
    pthread_t thread1, thread2;

    printf("=== SIMULASI DEADLOCK ===\n\n");

    // Membuat dua thread
    pthread_create(&thread1, NULL, thread1_func, NULL);
    pthread_create(&thread2, NULL, thread2_func, NULL);

    // Menunggu thread selesai (tidak akan pernah selesai karena deadlock)
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("\n=== PROGRAM SELESAI ===\n");

    return 0;
}

```

Cara Menjalankan:

```

# Compile program
gcc deadlock_simple.c -o deadlock_simple -lpthread

# Jalankan program
./deadlock_simple

```

Yang Akan Terjadi:

Program akan **HANG** (macet) dan tidak akan selesai. Untuk menghentikan, tekan **Ctrl+C**.

Analisis:

- Thread 1 pegang Lock 1, butuh Lock 2
 - Thread 2 pegang Lock 2, butuh Lock 1
 - Keduanya menunggu selamanya → **DEADLOCK!**
-

Topik 2: Deteksi Deadlock

Deadlock dengan Timeout

File: **deadlock_timeout.c**

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

int deadlock_detected = 0;

void* thread1_func(void* arg) {
    struct timespec timeout;
    int result;

    printf("Thread 1: Mengambil Lock 1\n");
    pthread_mutex_lock(&lock1);

    sleep(1);

    printf("Thread 1: Mencoba mengambil Lock 2 (dengan timeout 2 detik)...\n");

    // Set timeout 2 detik
    clock_gettime(CLOCK_REALTIME, &timeout);
    timeout.tv_sec += 2;

    result = pthread_mutex_timedlock(&lock2, &timeout);

    if (result != 0) {
        printf("Thread 1: TIMEOUT! Kemungkinan terjadi deadlock.\n");
        deadlock_detected = 1;
        pthread_mutex_unlock(&lock1);
        return NULL;
    }

    printf("Thread 1: Berhasil dapat kedua lock\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

void* thread2_func(void* arg) {
    printf("Thread 2: Mengambil Lock 2\n");
    pthread_mutex_lock(&lock2);
```

```

    sleep(1);

    printf("Thread 2: Mencoba mengambil Lock 1...\n");
    pthread_mutex_lock(&lock1);

    printf("Thread 2: Berhasil dapat kedua lock\n");

    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);

    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    printf("=== DETEKSI DEADLOCK DENGAN TIMEOUT ===\n\n");

    pthread_create(&thread1, NULL, thread1_func, NULL);
    pthread_create(&thread2, NULL, thread2_func, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    if (deadlock_detected) {
        printf("\n[SISTEM] Deadlock terdeteksi dan ditangani!\n");
    } else {
        printf("\n[SISTEM] Program berjalan normal.\n");
    }

    return 0;
}

```

Cara Menjalankan:

```

gcc deadlock_timeout.c -o deadlock_timeout -lpthread
./deadlock_timeout

```

Analisis:

Program ini mendeteksi deadlock menggunakan **timeout**. Jika lock tidak didapat dalam 2 detik, thread akan berhenti mencoba.

Topik 3: Pencegahan Deadlock

Teknik 1: Lock Ordering (Mencegah Circular Wait)

File: **prevention_ordering.c**

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

// Fungsi helper untuk mengambil lock dengan urutan yang sama
void acquire_locks_in_order() {
    // SELALU ambil lock1 dulu, baru lock2
    printf("  → Mengambil Lock 1\n");
    pthread_mutex_lock(&lock1);

    printf("  → Mengambil Lock 2\n");
    pthread_mutex_lock(&lock2);
}

void release_locks_in_order() {
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    printf("  → Lock dilepas\n");
}

void* thread1_func(void* arg) {
    printf("Thread 1: Mulai bekerja\n");

    acquire_locks_in_order();

    printf("Thread 1: Sedang menggunakan resource\n");
    sleep(1);

    release_locks_in_order();

    printf("Thread 1: Selesai\n");
    return NULL;
}

void* thread2_func(void* arg) {
    printf("Thread 2: Mulai bekerja\n");

    acquire_locks_in_order();

    printf("Thread 2: Sedang menggunakan resource\n");
    sleep(1);

    release_locks_in_order();
}
```

```

    printf("Thread 2: Selesai\n");
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    printf("=== PENCEGAHAN DEADLOCK: LOCK ORDERING ===\n\n");

    pthread_create(&thread1, NULL, thread1_func, NULL);
    pthread_create(&thread2, NULL, thread2_func, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("\n=== PROGRAM SELESAI TANPA DEADLOCK ===\n");

    return 0;
}

```

Cara Menjalankan:

```

gcc prevention_ordering.c -o prevention_ordering -lpthread
./prevention_ordering

```

Analisis:

Dengan memastikan **kedua thread mengambil lock dengan urutan yang sama**, kita menghilangkan kondisi Circular Wait dan mencegah deadlock.

Teknik 2: Try-Lock (Mencegah Hold and Wait)

File: prevention_trylock.c

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void* thread1_func(void* arg) {
    int retry_count = 0;

    while(1) {
        printf("Thread 1: Mencoba mengambil Lock 1\n");
        pthread_mutex_lock(&lock1);
    }
}

```

```

printf("Thread 1: Mencoba mengambil Lock 2 (try-lock)\n");

if (pthread_mutex_trylock(&lock2) == 0) {
    // Berhasil dapat kedua lock
    printf("Thread 1: ✓ Berhasil dapat kedua lock!\n");

    sleep(1); // Simulasi kerja

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    break;
} else {
    // Gagal dapat lock2, lepas lock1 dan coba lagi
    printf("Thread 1: X Gagal dapat Lock 2, melepas Lock 1\n");
    pthread_mutex_unlock(&lock1);
    retry_count++;

    usleep(100000); // Tunggu 0.1 detik sebelum retry
}
}

printf("Thread 1: Selesai (retry: %d kali)\n", retry_count);
return NULL;
}

void* thread2_func(void* arg) {
    int retry_count = 0;

    while(1) {
        printf("Thread 2: Mencoba mengambil Lock 2\n");
        pthread_mutex_lock(&lock2);

        printf("Thread 2: Mencoba mengambil Lock 1 (try-lock)\n");

        if (pthread_mutex_trylock(&lock1) == 0) {
            // Berhasil dapat kedua lock
            printf("Thread 2: ✓ Berhasil dapat kedua lock!\n");

            sleep(1); // Simulasi kerja

            pthread_mutex_unlock(&lock1);
            pthread_mutex_unlock(&lock2);
            break;
        } else {
            // Gagal dapat lock1, lepas lock2 dan coba lagi
            printf("Thread 2: X Gagal dapat Lock 1, melepas Lock 2\n");
            pthread_mutex_unlock(&lock2);
            retry_count++;

            usleep(150000); // Tunggu 0.15 detik sebelum retry
        }
    }
}

```



```

    printf("Thread 2: Selesai (retry: %d kali)\n", retry_count);
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    printf("=== PENCEGAHAN DEADLOCK: TRY-LOCK ===\n\n");

    pthread_create(&thread1, NULL, thread1_func, NULL);
    pthread_create(&thread2, NULL, thread2_func, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("\n=== PROGRAM SELESAI TANPA DEADLOCK ===\n");

    return 0;
}

```

Cara Menjalankan:

```

gcc prevention_trylock.c -o prevention_trylock -lpthread
./prevention_trylock

```

Analisis:

Teknik **try-lock** mencoba mengambil lock, jika gagal maka thread akan melepas semua lock yang sudah dipegang dan mencoba lagi. Ini mencegah kondisi Hold and Wait.

Topik 4: Banker's Algorithm (Menghindari Deadlock)

Simulasi Banker's Algorithm

File: **bankers_algorithm.c**

```
#include <stdio.h>
#include <stdbool.h>

#define P 5 // Jumlah proses
#define R 3 // Jumlah jenis resource

// Data sistem
int available[R] = {3, 3, 2}; // Resource yang tersedia

int maximum[P][R] = { // Kebutuhan maksimum setiap proses
    {7, 5, 3},
    {3, 2, 2},
    {9, 0, 2},
    {2, 2, 2},
    {4, 3, 3}
};

int allocation[P][R] = { // Resource yang sudah dialokasikan
    {0, 1, 0},
    {2, 0, 0},
    {3, 0, 2},
    {2, 1, 1},
    {0, 0, 2}
};

int need[P][R]; // Kebutuhan yang tersisa

// Fungsi untuk menghitung kebutuhan
void calculate_need() {
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }
}

// Fungsi untuk mencetak status sistem
void print_status() {
    printf("\n=== STATUS SISTEM ===\n");
    printf("\nAvailable: ");
    for (int i = 0; i < R; i++) printf("%d ", available[i]);

    printf("\n\nAllocation:\n");
    for (int i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (int j = 0; j < R; j++) printf("%d ", allocation[i][j]);
    }
}
```

```

        printf("\n");
    }

    printf("\nNeed:\n");
    for (int i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (int j = 0; j < R; j++) printf("%d ", need[i][j]);
        printf("\n");
    }
}

// Fungsi untuk mengecek apakah sistem dalam keadaan aman
bool is_safe() {
    int work[R];
    bool finish[P] = {false};
    int safe_sequence[P];
    int count = 0;

    // Copy available ke work
    for (int i = 0; i < R; i++) {
        work[i] = available[i];
    }

    printf("\n=== MEMERIKSA KEAMANAN SISTEM ===\n");

    // Cari urutan aman
    while (count < P) {
        bool found = false;

        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                bool can_allocate = true;

                // Cek apakah need[i] <= work
                for (int j = 0; j < R; j++) {
                    if (need[i][j] > work[j]) {
                        can_allocate = false;
                        break;
                    }
                }

                if (can_allocate) {
                    printf("P%d dapat dijalankan. ", i);
                    printf("Work: [");
                    for (int j = 0; j < R; j++) printf("%d ", work[j]);
                    printf("]\n");

                    // Simulasi proses selesai
                    for (int j = 0; j < R; j++) {
                        work[j] += allocation[i][j];
                    }

                    safe_sequence[count++] = i;
                    finish[i] = true;
                }
            }
        }
    }
}

```

```

        found = true;
    }
}

if (!found) {
    printf("\n[PERINGATAN] Tidak dapat menemukan urutan aman!\n");
    return false;
}

printf("\n[SUKSES] Sistem dalam keadaan AMAN!\n");
printf("Safe Sequence: ");
for (int i = 0; i < P; i++) {
    printf("P%d ", safe_sequence[i]);
    if (i < P - 1) printf("→ ");
}
printf("\n");

return true;
}

// Fungsi untuk meminta resource
bool request_resources(int process, int request[]) {
    printf("\n=== REQUEST DARI P%d ===\n", process);
    printf("Meminta: ");
    for (int i = 0; i < R; i++) printf("%d ", request[i]);
    printf("\n");

    // Cek apakah request <= need
    for (int i = 0; i < R; i++) {
        if (request[i] > need[process][i]) {
            printf("[DITOLAK] Request melebihi kebutuhan maksimum!\n");
            return false;
        }
    }

    // Cek apakah request <= available
    for (int i = 0; i < R; i++) {
        if (request[i] > available[i]) {
            printf("[DITOLAK] Resource tidak tersedia!\n");
            return false;
        }
    }

    // Simulasi alokasi
    for (int i = 0; i < R; i++) {
        available[i] -= request[i];
        allocation[process][i] += request[i];
        need[process][i] -= request[i];
    }

    // Cek keamanan
    if (is_safe()) {

```

```

        printf("[DITERIMA] Request berhasil dialokasikan!\n");
        return true;
    } else {
        // Rollback
        printf("[DITOLAK] Request menyebabkan unsafe state!\n");
        for (int i = 0; i < R; i++) {
            available[i] += request[i];
            allocation[process][i] -= request[i];
            need[process][i] += request[i];
        }
        return false;
    }
}

int main() {
    printf("=== BANKER'S ALGORITHM ===\n");

    calculate_need();
    print_status();

    // Cek keadaan awal
    is_safe();

    // Simulasi request 1 (aman)
    int request1[R] = {1, 0, 2};
    request_resources(1, request1);
    print_status();

    // Simulasi request 2 (tidak aman)
    int request2[R] = {3, 3, 0};
    request_resources(4, request2);

    printf("\n=== PROGRAM SELESAI ===\n");
    return 0;
}

```

Cara Menjalankan:

```

gcc bankers_algorithm.c -o bankers_algorithm
./bankers_algorithm

```

Analisis:

Banker's Algorithm mencegah deadlock dengan hanya memberikan resource jika sistem tetap dalam **safe state** setelah alokasi.

Laporan Praktikum

Silakan buat dokumentasi praktikum seperti biasanya. Pastikan pada laporan praktikum mencantumkan identitas, kode program, output kode program, dan analisis secara umum makna dari kode program tersebut disertai penjelasan sintaks kode program tertentu yang berpengaruh dari konteks kode program yang Anda ujicoba. Tidak semua sintaks harus dijelaskan.

Tugas Praktikum

Tugas 1: Analisis Deadlock

Jalankan program `deadlock_simple.c` dan jelaskan:

1. Mengapa program mengalami deadlock?
2. Kondisi deadlock apa saja yang terpenuhi?
3. Gambar diagram resource allocation graph-nya

Tugas 2: Implementasi Pencegahan

Modifikasi program `deadlock_simple.c` menggunakan teknik:

1. Lock ordering
2. Try-lock dengan retry
3. Bandingkan kedua metode tersebut

Tugas 3: Banker's Algorithm

Buat program baru dengan data:

- 4 proses
- 3 jenis resource
- Available: [5, 4, 3]
- Implementasikan request resource dan cek keamanan sistem

Tugas 4: Kasus Nyata (Real Case)

Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"

- Orang A: transfer dari Rekening 1 ke Rekening 2
 - Orang B: transfer dari Rekening 2 ke Rekening 1
 - Implementasikan deadlock prevention
-

Tips Debugging

Cara Melihat Thread yang Hang:

```
# Jalankan program di background
./deadlock_simple &

# Lihat status thread
ps -eLf | grep deadlock_simple

# Atau gunakan top
top -H -p [PID]
```

Menggunakan GDB untuk Debug Deadlock:

```
# Compile dengan debug info
gcc -g deadlock_simple.c -o deadlock_simple -lpthread

# Jalankan dengan GDB
gdb ./deadlock_simple

# Di dalam GDB
(gdb) run
# Ketika hang, tekan Ctrl+C
(gdb) info threads
(gdb) thread 2
(gdb) backtrace
```

Referensi

- Manual page: `man pthread_mutex_lock`
- Manual page: `man pthread_mutex_trylock`
- Operating System Concepts (Silberschatz)
- POSIX Threads Programming: <https://hpc-tutorials.llnl.gov/posix/>